A reproducible federated anomaly detection pipeline for resource-constrained IIoT clients under non-IID and Byzantine conditions

Sundetkhan Bekzat, Baibolat Bekarys

Abstract. Federated anomaly detection is attractive for industrial Internet of Things deployments because raw sensor streams remain on the device, but the resulting training process must operate under three constraints at once: limited uplink capacity, non-IID client data, and the possibility of poisoned updates. This article describes a reproducible pipeline that combines a local LSTM autoencoder, Top-k error-feedback compression, FedProx stabilisation, and Byzantine-robust server aggregation in one round-level workflow. The implementation couples local reconstruction-based scoring with compressed transmission, robust aggregation, global model broadcast, and client-side thresholding. Validation is carried out on a five-client synthetic IIoT benchmark with heterogeneous periodicities, amplitudes, noise levels, and injected anomalies. The pipeline reduces communication from 14.92 MB to 1.49 MB at the 10% compression setting while preserving similar AUROC to the uncompressed communication study, and it remains markedly more stable than FedAvg when malicious clients are introduced. The method is intended as a compact reference implementation that other researchers can reproduce, adapt, and extend when studying federated anomaly detection for constrained industrial edge settings.

Keywords: federated learning; anomaly detection; industrial IoT; non-IID learning; Byzantine robustness; gradient compression

1 Introduction

Industrial edge systems increasingly place analytics close to sensors, controllers, and gateways because continuous transfer of multivariate process data is often impractical. In anomaly detection, this architectural shift creates a familiar tension. Local models reduce data movement and may protect sensitive operational traces, yet isolated training wastes information that is distributed across many devices. Federated learning addresses this tension by exchanging model updates instead of raw data, but standard federated optimisation is poorly matched to realistic industrial conditions. The communication budget is tight, client distributions differ across assets and operating regimes, and a compromised participant can poison the global model.

The present work addresses these constraints as one implementation problem rather than as three separate add-ons. The method integrates a reconstruction-based local anomaly detector with compressed federated communication, non-IID stabilisation, and Byzantine-robust aggregation. The intent is methodological. The article does not propose a new anomaly score or a new robust estimator in isolation. Instead, it documents how these components can be assembled into a single reproducible training and inference pipeline for resource-constrained IIoT clients.

This positioning is important for a MethodsX article. What is customised here is the operational coupling of four modules that are often discussed independently: the local anomaly detection model, the communication-efficient federated protocol, the non-IID stabilisation mechanism, and the robust server-side defence. The article therefore focuses on implementation details, interfaces between modules, and the trade-offs that arise when the full system is exercised under heterogeneous and adversarial conditions.

2 Method Positioning

Federated learning for edge analytics remains anchored by the communication argument introduced by FedAvg [1], but later work makes clear that communication reduction alone is not enough once data heterogeneity and adversarial behaviour are taken seriously. FedProx [2], SCAFFOLD [4], Ditto [5], and pFedMe [6] all target instability induced by client drift, while FedPAQ [3] and sparsification methods [19] reduce message size or synchronisation frequency. In parallel, Byzantine-robust aggregation has produced a family of defences including Krum [7], trimmed mean and coordinate-wise median [9], Bulyan [8], RFA [10], and trust-based approaches such as FLTrust [11].

For anomaly detection, reconstruction-based sequence models remain attractive because they map naturally onto unsupervised industrial monitoring. LSTM autoencoders and related architectures continue to be used in multivariate time-series settings where labelled attack data are scarce [15, 18, 20]. The gap addressed in this article is therefore not the absence of candidate components. It lies in the lack of a compact, reproducible workflow that shows how these components interact when a practitioner must train an anomaly detector under bandwidth limits, non-IID clients, and poisoned updates at the same time.

3 Problem Setup

We consider a federation of K edge clients $\mathcal{K} = \{1, ..., K\}$. Client k stores a local multivariate time-series dataset D_k with n_k samples generated from a client-specific distribution $\mathcal{P}_k$. The global objective is

$$\min_{\theta} F(\theta) = \sum_{k=1}^{K} p_k F_{k(\theta)}, \quad F_{k(\theta)} = \frac{1}{n_k} \sum_{i=1}^{n_k} \ell\Big(\boldsymbol{x}_i^{(k)}, \theta\Big), \quad (1)$$

where $p_k = \frac{n_k}{\sum_j} n_j$. The non-IID setting is expressed by $\mathcal{P}_k \neq \mathcal{P}_j$ for at least some client pairs. In practical terms, different industrial assets produce distinct frequencies, amplitudes, noise levels, and anomaly manifestations. A server coordinates training rounds, but raw windows are never uploaded. The adversarial model assumes that a fraction α of selected clients may transmit manipulated updates. The experiments cover model poisoning and on-off behaviour, where malicious clients alternate between benign and harmful rounds in an attempt to evade simple defences. The methodological objective is therefore to produce a training loop that is communication-aware, more stable under heterogeneous local objectives, and less sensitive to corrupted client updates.

4 Integrated Method

The proposed system is executed as one repeated workflow. Each client converts its local multivariate stream into sliding windows, trains an LSTM autoencoder for several local epochs, regularises the local objective with a FedProx term, compresses the resulting parameter update through Top-k error-feedback sparsification, and transmits the compressed update to the server. The server aggregates incoming updates with a robust rule, updates the global model, and broadcasts the revised parameters back to clients. Each client then applies a local threshold to reconstruction errors for inference. The complete flow is summarized in Figure 1 and in Table 1.

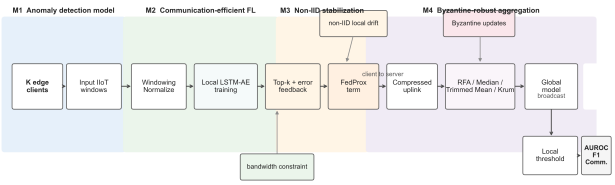

Figure 1: Integrated overview of the proposed federated anomaly detection pipeline for IIoT edge networks. Local clients process multivariate sensor windows, train an LSTM autoencoder with FedProx stabilisation, compress updates with Top-k error-feedback, and transmit them to a server that performs Byzantine-robust aggregation before broadcasting the updated global model. The pipeline links the four method modules in a single end-to-end workflow from raw data to local inference and evaluation.

Module	Role in the pipeline	Implementation in this article	Expected effect
M1: Local detector	Construct sample-level anomaly scores from local windows	Single-layer LSTM autoencoder with reconstruction loss and client-side percentile threshold	Compact unsupervised detector that remains deployable on edge hardware
M2: Communication-efficient FL	Reduce uplink cost during federated training	Top-k sparsification with error-feedback, partial participation, and multi-epoch local updates	Lower communication without transmitting full updates each round
M3: Non-IID stabilisation	Limit client drift under heterogeneous local distributions	FedProx proximal regularisation plus local thresholding at inference time	Smoother optimisation and client-specific decision boundaries
M4: Byzantine defence	Reduce the impact of corrupted local updates	RFA as the main robust aggregator, with median, trimmed mean, and Krum as comparison baselines	Greater resistance to model poisoning than vanilla averaging

Table 1: Synthesis of the four modules used in the integrated pipeline.

4.1 Local anomaly detection model

The base detector is an LSTM autoencoder that operates on sliding windows $\boldsymbol{X} \in \mathbb{R}^{W \times F}$, where W is window length and F is the number of sensor channels. The encoder maps the input window to a latent representation and the decoder reconstructs the original sequence,

$$\boldsymbol{z} = \text{Encoder}_{\theta(\boldsymbol{X})} \in \mathbb{R}^{L}, \quad \hat{\boldsymbol{X}} = \text{Decoder}_{\varphi(\boldsymbol{z})} \in \mathbb{R}^{W \times F}. \tag{2}$$

Training minimises the mean squared reconstruction error,

$$\ell(\boldsymbol{X}, \theta, \varphi) = \frac{1}{WF} \left\| \boldsymbol{X} - \hat{\boldsymbol{X}} \right\|_F^2. \tag{3}$$

At inference, the anomaly score is the reconstruction loss itself. A client flags a window as anomalous when the score exceeds a percentile threshold estimated from local training data. This design preserves a shared representation across the federation while allowing decision thresholds to remain client-specific, which is useful when baseline operating ranges differ across assets.

The network is intentionally compact. The encoder uses one LSTM layer with hidden size $H = 48$, followed by projection to latent size $L = 16$ in the manuscript formulation. The implementation configuration stored with the experiments uses latent size 12, as recorded in the saved run configuration, and the code follows that setting for reproducibility. The decoder repeats the latent vector over the input horizon and reconstructs the full multivariate window. The resulting model remains small enough to illustrate the communication effects of federated training without overwhelming them with model size alone.

4.2 Communication-efficient federated update

After E local epochs, client k computes an update relative to the current global model,

$$\Delta_k^t = \theta_k^t - \theta_{\text{global}}^t. \tag{4}$$

To reduce message size, the client applies Top-k sparsification with error feedback,

$$\Delta_k^t + e_k^t \to C_{k(\Delta_k^t + e_k^t, \kappa)}, \quad e_k^{t+1} = (\Delta_k^t + e_k^t) - C_{k(\Delta_k^t + e_k^t, \kappa)}, \tag{5}$$

where $C_{k(\Delta, \kappa)}$ denotes the compression operator that retains the fraction κ of components with the largest absolute magnitude. The residual e_k^t accumulates the information removed in previous rounds and re-injects it later, reducing the bias that would otherwise be introduced by repeated sparsification. The experiments sweep $\kappa \in \{1\%, 5\%, 10\%, 20\%, 50\%, 100\%\}$ and use 10% as the default operating point for the integrated pipeline.

Only a subset of clients participates in each round. In the implementation, the server samples $C = 0.6$ of the client pool per round and each selected client performs $E = 3$ local epochs before transmission. This choice follows the logic of communication-efficient federated learning: fewer synchronisation events and smaller updates both reduce total traffic. The article treats this communication module as part of the complete anomaly detection workflow rather than as an isolated compression experiment.

4.3 Non-IID stabilisation and local decision adaptation

Under heterogeneous client distributions, local optimisation may drift away from a globally useful parameter region. To reduce this effect, local training includes the FedProx objective,

$$\min_{\theta_k} \left\{ F_{k(\theta_k)} + \frac{\mu}{2} \left\| \theta_k - \theta_{\text{global}}^t \right\|^2 \right\}, \tag{6}$$

with proximal coefficient $\mu = 0.01$. The proximal term discourages large departures from the broadcast model, which is especially relevant when some clients observe extreme

amplitudes or noisier operating conditions. In this pipeline, FedProx interacts directly with the other modules: it helps stabilise the update that will later be sparsified and robustly aggregated, and it reduces the chance that heterogeneous local steps are mistaken for adversarial behaviour.

Inference retains a local component even though training is federated. Each client sets its own threshold on reconstruction error using a percentile of local training scores. This choice is pragmatic. It keeps the representation shared, but avoids forcing a single global threshold onto devices that operate in different normal regimes.

4.4 Byzantine-robust server aggregation

Given the set of compressed updates $\{\hat{\Delta}_k^t\}_{k\in\mathcal{S}_t}$ received from the selected client set $\mathcal{S}_t$, the server updates the global model with a robust aggregation rule. The main implementation uses robust federated aggregation (RFA), which approximates the geometric median through Weiszfeld iterations,

$$\Delta^{t+1} = \sum_k w_k \hat{\Delta}_k^t, \quad w_k = \frac{1}{\max\left(\left\|\hat{\Delta}_k^t - \Delta^{(i)}\right\|, \varepsilon\right)}, \tag{7}$$

initialised from the arithmetic mean and iterated five times. The appeal of RFA in this setting is that it can damp the influence of outlying updates while remaining compatible with compressed transmission.

The evaluation also includes coordinate-wise median, trimmed mean, and Krum. These baselines are useful because they respond differently to heterogeneity and poisoning. Median operates coordinate-wise and often handles dispersed outliers well. Trimmed mean is less aggressive, which can be advantageous when honest updates are not too widely spread. Krum relies on distance-based neighbour selection and can be brittle when honest updates are already dispersed by non-IID data. Presenting these methods within the same pipeline makes the design choices easier to compare under one workload.

5 Experimental Setup

The synthetic benchmark is designed to expose the method to realistic heterogeneity while remaining fully reproducible. Five clients are generated, each representing a distinct industrial node with client-specific frequency, amplitude, and noise parameters. For feature f at client k, the signal is generated as

$$x_f^{(k)}(t) = A_k\big(\sin(2\pi f_k t + \varphi_f) + h_k \sin(4\pi f_k t + \varphi_f)\big) + \varepsilon_f^{(k)}(t) + b_f^{(k)}, \tag{8}$$

where $f_k = 0.01 + 0.008k$, $A_k = 1 + 0.4k$, $\varepsilon_f^{(k)}(t) \sim \mathcal{N}(0, \sigma_k^2)$, and $b_f^{(k)}$ is a feature-specific offset. Each client contributes 4000 samples over six channels. The dataset is not intended as a replacement for established industrial control system benchmarks. Its purpose is different: it provides a transparent benchmark in which the sources of heterogeneity and anomaly injection are known and controllable.

Anomalies are injected at an approximate overall rate of 6%. The benchmark mixes point spikes, drift segments, and pattern shifts so that the anomaly detector is not evaluated on a single failure mode. Train and test partitions follow a 70/30 split. Training remains unsupervised because only windows ending in normal states are retained for local model fitting, while the test partition keeps anomalous windows.

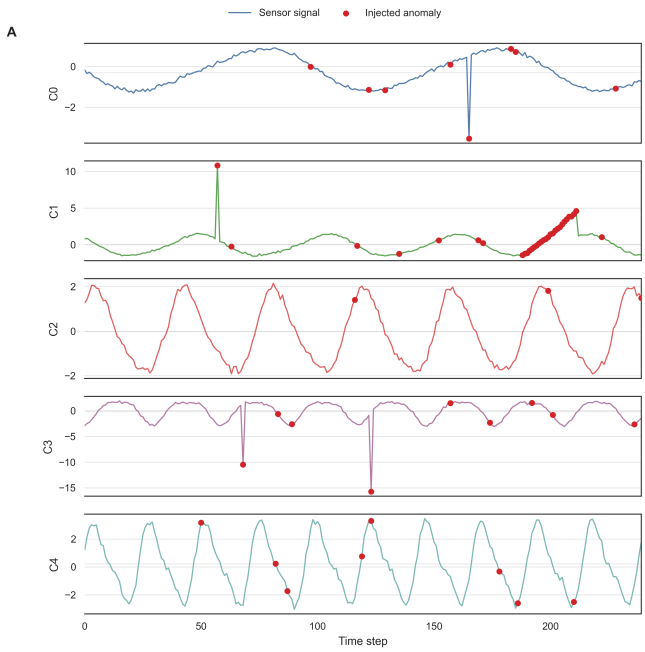

Figure 2: Overview of the synthetic five-client IIoT dataset used in the federated experiments. The clients show different frequencies, amplitudes, and noise levels, illustrating the non-IID setting, while red markers indicate injected anomalies including spikes, drift segments, and pattern shifts.

The implementation uses PyTorch 2.11, Adam with learning rate 10^{-3}, batch size 64, local epoch count $E = 3$, and 40 to 60 communication rounds depending on the experiment. The stored configuration records a latent size of 12 for the experimental run, whereas the method description above retains the 16-dimensional formulation used in the conceptual design. This discrepancy is reported explicitly here because reproducibility is more important than cosmetic consistency. All random seeds are fixed to 42.

The attack experiments vary the fraction of malicious clients over $\alpha \in \{0, 0.1, 0.2, 0.3\}$. Model poisoning is implemented by transmitting harmful updates, and the on-off setting alternates malicious and benign behaviour across rounds. Evaluation reports AUROC, F1, and communication volume in megabytes computed from the transmitted update size.

6 Results and Interpretation

6.1 Convergence behaviour and communication burden

Figure 3 compares four federated configurations. FedProx produces smoother optimisation trajectories than plain FedAvg, which is consistent with the role of the proximal term under heterogeneous local objectives. The benefit appears mainly in stability rather than in the final metric alone: the loss curves fluctuate less strongly once local training begins to separate client behaviour. Compression changes a different part of the pipeline. Top-k cuts cumulative communication by roughly one order of magnitude because only 10% of update coordinates are transmitted, yet the learning curves remain in a comparable range. This suggests that, for the present model size and benchmark, most

communication can be removed before the detector quality deteriorates sharply.

The combined FedProx + Top-k configuration is therefore best understood as a stability-efficiency compromise rather than a uniformly dominant setting. Its final AUROC in the convergence experiment is lower than that of uncompressed FedAvg, but the communication cost falls from 22.37 MB to 2.24 MB over 60 rounds. For an edge deployment, that trade-off can still be attractive because uplink constraints are often hard rather than optional.

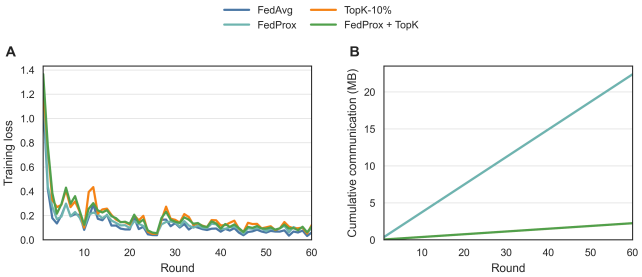

Figure 3: Training loss and cumulative communication over federated rounds for FedAvg, FedProx, Top-k, and FedProx + Top-k. FedProx yields smoother convergence under non-IID data, while Top-k strongly reduces communication; the combined configuration preserves convergence behaviour with much lower uplink cost.

6.2 Compression ratio versus detection quality

The dedicated communication sweep in Figure 4 isolates the effect of sparsification. Moving from full transmission to the 10% setting reduces communication from 14.92 MB to 1.49 MB, while AUROC changes from 0.663 to 0.658. The drop is therefore small relative to the communication saving. More aggressive compression reveals the expected trade-off. At 1% transmission, the total payload falls to 0.149 MB, but AUROC declines to 0.632. Error feedback clearly delays degradation, yet it does not eliminate it.

This pattern matters operationally. In IIoT deployments, the preferred compression ratio is not the one with the best metric in isolation, but the one that gives an acceptable loss in detection quality for a substantial reduction in traffic. On the present benchmark, the 10% ratio is the most defensible default because it removes most communication while avoiding the sharper performance loss observed at 1%.

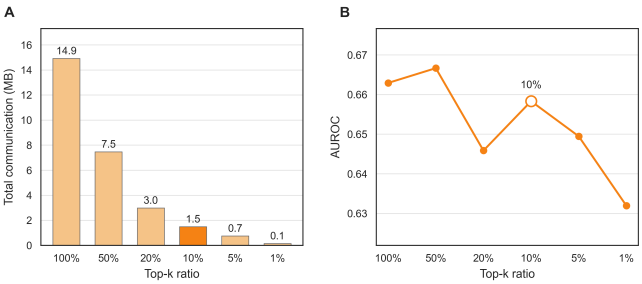

Figure 4: Communication cost and detection quality across Top-k compression ratios from 100% to 1%. Uplink volume decreases nearly linearly with stronger compression, while AUROC changes modestly; the 10% setting provides a compact trade-off between communication reduction and detection performance.

6.3 Example anomaly detection output

The qualitative example in Figure 5 shows how the local inference stage behaves on a representative client. Reconstruction error rises around anomalous intervals and stays lower during normal segments, which is the expected behaviour for a reconstruction-based detector trained on normal windows only. The figure is not meant as a substitute for the quantitative evaluation. Its value lies in making the final stage of the pipeline explicit: the federated model is still used locally, and the final decision remains a thresholding problem on client-side scores.

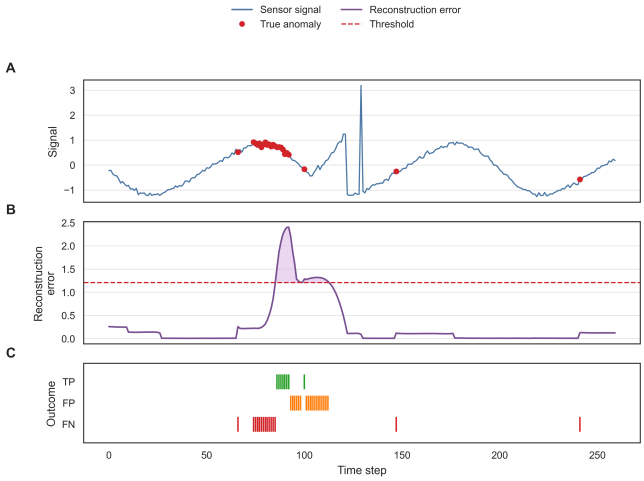

Figure 5: Representative anomaly detection output for Client 0. The top panel shows the sensor signal and true anomalies, the middle panel shows reconstruction error with the decision threshold, and the bottom panel summarises detection outcomes. Elevated reconstruction error aligns with anomalous intervals, with relatively few false detections in normal regions.

6.4 Robustness under Byzantine updates

The most informative robustness pattern in Figure 6 is not simply that FedAvg degrades, but how quickly the degradation appears once malicious clients are introduced. FedAvg falls from AUROC 0.685 at 10% malicious participation to 0.477 at 20%, which indicates that the combination of non-IID dispersion and poisoning is enough to push mean aggregation into failure. Median behaves differently. Its AUROC increases from 0.741 without attack to 0.804 at 30% malicious clients on this benchmark. That counterintuitive behaviour should not be interpreted as a universal gain from more attackers. It instead suggests that the coordinate-wise median is strongly dampening outlying updates, including some variability introduced by honest heterogeneity.

RFA is also substantially more stable than FedAvg, reaching AUROC about 0.703 at both 20% and 30% attack fractions. It does not outperform median in these runs, but it remains more robust than FedAvg and more stable than Krum. Krum performs worse than the stronger robust baselines, which is consistent with distance-based selection becoming less reliable when honest client updates are already spread by non-IID data. The practical implication is that robust aggregation should be treated as essential once the method is deployed in adversarial settings, but the specific choice of aggregator still depends on how strongly heterogeneity disperses honest updates.

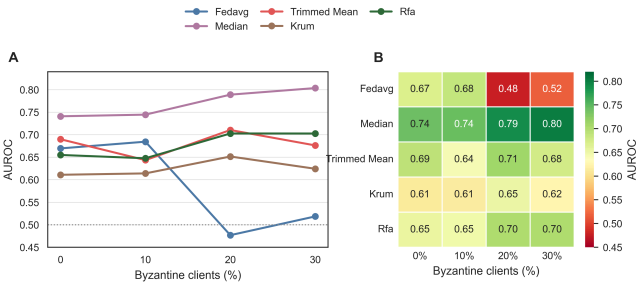

Figure 6: Robustness of aggregation methods under increasing fractions of Byzantine clients. The line plot and heatmap report AUROC for FedAvg, median, trimmed mean, Krum, and RFA across attack fractions from 0% to 30%. FedAvg degrades sharply at higher attack rates, whereas median and RFA remain substantially more stable.

6.5 On-off attack behaviour

The on-off experiment in Figure 7 clarifies that evasive attack patterns do not automatically break robust methods, but they do change method ranking less than expected in this benchmark. Median is the strongest performer under both persistent poisoning and on-off behaviour, with AUROC above 0.80 in both cases. RFA remains second, while FedAvg stays far behind. Interestingly, on-off behaviour slightly improves FedAvg and median relative to persistent poisoning, but it reduces RFA from 0.718 to 0.694. The likely explanation is that alternating behaviour softens the extremeness of some poisoned updates, which can occasionally make them less visible to geometric-median style filtering while still leaving the median comparatively stable. This result is a useful caution against generic claims about evasive attacks. The effect of on-off behaviour depends on the interaction between attack strength, honest heterogeneity, and the geometry of the aggregation rule. For this dataset, the main conclusion is not that on-off attacks defeat robust aggregation, but that they change the stress pattern enough to justify explicit evaluation rather than assuming that persistent poisoning is always the hardest case.

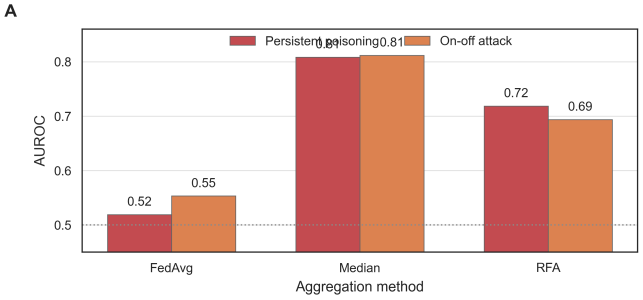

Figure 7: AUROC under persistent model poisoning and on-off evasion attacks with 20% adversarial clients. Median and RFA remain clearly stronger than FedAvg under both attack patterns, while on-off behaviour changes performance only modestly relative to persistent poisoning in this experiment.

6.6 Ablation of the integrated pipeline

The ablation experiment is the clearest test of whether the four modules contribute distinct value. The answer is yes, but not in the same way. Removing robust aggregation has the largest impact on detection quality under attack, lowering AUROC to 0.460 and F1 to 0.062 despite keeping communication low. This makes the defence module indispensable in the adversarial setting considered here. Removing FedProx also lowers AUROC, although the effect is more moderate, which matches the role of FedProx as a stabiliser rather than a primary defence.

Compression behaves differently. The uncompressed FedProx + RFA variant yields slightly better AUROC and F1 than the compressed full system, but it does so at ten times the communication cost. The integrated pipeline should therefore not be described as the best variant on every single metric. A more accurate interpretation is that it offers the best balance among communication cost, detection quality, and attack resilience. That is the reason it is retained as the recommended reference configuration.

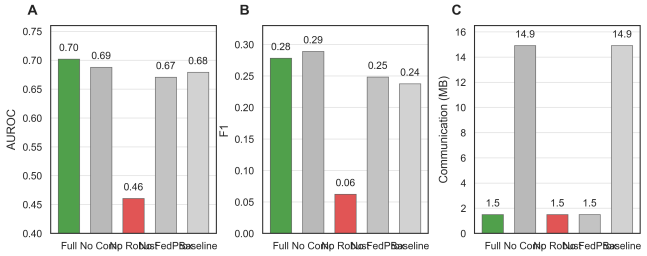

Figure 8: Ablation analysis of the proposed system under 20% model-poisoning attackers. AUROC, F1, and total communication are shown for the full method and four reduced variants. Removing robust aggregation causes the strongest performance drop, whereas removing compression mainly increases communication cost.

Configuration	AUROC	F1	Comm. (MB)
Full system (FedProx + Top-k + RFA)	0.702	0.278	1.49
No compression (FedProx + RFA)	0.688	0.289	14.92
No robust aggregation (FedProx + Top-k)	0.460	0.062	1.49
No FedProx (FedAvg + Top-k + RFA)	0.671	0.248	1.49
Baseline (FedAvg only)	0.679	0.238	14.92

Table 2: Numerical summary of the ablation study under 20% model-poisoning attackers.

7 Discussion

The results show that communication efficiency, heterogeneity handling, and robustness cannot be tuned independently. Compression is most convincing when the local optimiser is already stabilised, because aggressive sparsification amplifies noise in drifting client updates. This helps explain why a simple communication study can look favourable at 10% sparsification while the integrated system still requires careful balancing once adversaries are introduced.

The comparison among robust aggregators also deserves a restrained reading. Median is the strongest method on the stored robustness and on-off experiments, but that does not make it universally preferable. Coordinate-wise aggregation can behave well when the attack produces clear outliers, yet it ignores cross-coordinate structure. RFA is attractive because it remains strong across several attack levels and fits naturally into the compressed update pipeline. Krum is more fragile in this study, which is plausible because distance-based neighbour selection becomes difficult when honest updates are already dispersed by non-IID client distributions.

The synthetic benchmark supports controlled methodological comparison, but its interpretation has limits. It

captures heterogeneity through client-specific frequencies, amplitudes, noise, and anomaly types, yet it does not reproduce the process dependencies, actuation loops, or operator interventions present in real plants. The method should therefore be viewed as a reproducible reference pipeline rather than as a claim of deployment readiness on any specific industrial control system. Transfer to real settings is most plausible for the structural lessons: local thresholding remains useful under heterogeneous operating regimes, communication can often be reduced sharply before quality collapses, and robust aggregation becomes important once malicious participation is credible.

The method may fail under several conditions. Very severe compression will eventually remove too much information even with error feedback. Strongly personalized client behaviour could make a single shared encoder inadequate. Adaptive adversaries that explicitly optimise against the selected robust rule may also erode performance. These are natural directions for extending the pipeline with adaptive thresholding, stronger personalization, or trust-based server-side checks.

8 Conclusion

This article documents a compact federated anomaly detection pipeline for constrained IIoT clients and shows how four standard ingredients can be integrated into one reproducible workflow: local LSTM autoencoder training, communication-efficient update compression, FedProx stabilisation, and Byzantine-robust aggregation. The empirical contribution is not a claim that one configuration dominates every metric, but that the integrated design gives a workable balance between communication cost, detection quality, and robustness. On the present benchmark, the 10% compression setting preserves similar AUROC to full transmission while reducing payload substantially, and robust aggregation is necessary once malicious updates are present.

The method is intended to be reusable. Another researcher can reproduce the benchmark generation process, the training loop, the compression schedule, the attack scenarios, and the evaluation workflow from the information reported here and from the accompanying code base. That reproducible integration, rather than any single standalone algorithmic novelty, is the central contribution of the article.

9 Code and Data Availability

The source code, generated benchmark, experiment scripts, and figure-generation utilities are available in the project repository: https://github.com/Bekzat158/multi-agent-system-assig. The synthetic dataset is generated procedurally from fixed random seeds and parameter settings reported in the manuscript and stored in `results/experiment_results.json`.

10 Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

11 Acknowledgements

The authors thank Seema Rawat for course supervision and feedback during the development of the project from which this method article was derived.

12 Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this article.

13 References

[**1**] McMahan, B., Moore, E., Ramage, D., Hampson, S., & Agüera y Arcas, B. (2017). Communication-efficient learning of deep networks from decentralized data. *AISTATS 2017*, PMLR 54, 1273–1282.

[**2**] Li, T., Sahu, A. K., Zaheer, M., Sanjabi, M., Talwalkar, A., & Smith, V. (2020). Federated optimization in heterogeneous networks. *MLSys 2020*.

[**3**] Reisizadeh, A., Mokhtari, A., Hassani, H., Jadbabaie, A., & Pedarsani, R. (2020). FedPAQ: A communication-efficient federated learning method with periodic averaging and quantization. *AISTATS 2020*, PMLR 108.

[**4**] Karimireddy, S. P., Kale, S., Mohri, M., Reddi, S. J., Stich, S. U., & Suresh, A. T. (2020). SCAFFOLD: Stochastic controlled averaging for federated learning. *ICML 2020*, PMLR 119.

[**5**] Li, T., Hu, S., Beirami, A., & Smith, V. (2021). Ditto: Fair and robust federated learning through personalization. *ICML 2021*, PMLR 139.

[**6**] Dinh, C. T., Tran, N. H., & Nguyen, T. D. (2020). Personalized federated learning with Moreau envelopes. *NeurIPS 2020*.

[**7**] Blanchard, P., Mhamdi, E. M. E., Guerraoui, R., & Stainer, J. (2017). Machine learning with adversaries: Byzantine tolerant gradient descent. *NeurIPS 2017*.

[**8**] El Mhamdi, E. M., Guerraoui, R., & Rouault, S. (2018). The hidden vulnerability of distributed learning in Byzantium. *ICML 2018*, PMLR 80.

[**9**] Yin, D., Chen, Y., Kannan, R., & Bartlett, P. (2018). Byzantine-robust distributed learning: Towards optimal statistical rates. *ICML 2018*, PMLR 80.

[**10**] Pillutla, K., Kakade, S. M., & Harchaoui, Z. (2022). Robust aggregation for federated learning. *IEEE Transactions on Signal Processing*, 70, 1142–1154.

[**11**] Cao, X., Fang, M., Liu, J., & Gong, N. Z. (2021). FLTrust: Byzantine-robust federated learning via trust bootstrapping. *NDSS 2021*.

[**12**] Bagdasaryan, E., Veit, A., Hua, Y., Estrin, D., & Shmatikov, V. (2020). How to backdoor federated learning. *AISTATS 2020*, PMLR 108.

[**13**] Fang, M., Cao, X., Jia, J., & Gong, N. Z. (2020). Local model poisoning attacks to Byzantine-robust federated learning. *USENIX Security 2020*.

[**14**] Xie, C., Chen, M., Chen, P.-Y., & Li, B. (2023). Attacks against federated learning defense systems and their implications. *JMLR*, 24(305), 1–43.

[**15**] Yuan, D., Hu, S., Guo, S., Zhang, J., & Yang, B. (2020). Deep anomaly detection for time-series data in industrial IoT: A communication-efficient on-device federated learning approach. *IEEE Internet of Things Journal*, 8(9), 6348–6358.

[**16**] Ruff, L., Vandermeulen, R., Goernitz, N., et al. (2018). Deep one-class classification. *ICML 2018*, PMLR 80.

[**17**] Zhang, C., Zhu, Y., Zhang, X., & Li, Z. (2022). Deep federated anomaly detection for multivariate time series data. *arXiv:2205.04571*.

[**18**] Audibert, J., Michiardi, P., Guyard, F., Marti, S., & Zuluaga, M. A. (2022). Deep learning for anomaly detection in time series: A survey. *ACM Computing Surveys*, 54(3).

[**19**] Alistarh, D., De Sa, C., & Shah, N. (2018). The convergence of sparsified gradient methods. *NeurIPS 2018*.

[**20**] Park, D., Hoshi, Y., & Kemp, C. C. (2018). A multimodal anomaly detector for robot-assisted feeding using an LSTM-based variational autoencoder. *IEEE Robotics and Automation Letters*, 4(2), 1543–1550.